PAGE 1 - MEMORIZE (common for all 8 questions)

X = class name from the question (Patient, Book, Employee ...). **x** = small first letter. Yellow = memorize.

1. pom.xml - inside <dependencies>

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

2. application.properties (change only DB name + password)

```
spring.datasource.url=jdbc:postgresql://localhost:5432/DBNAME
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

3. Entity X.java

```
@Entity
public class X {
    @Id
    private int xId;          <- fields from the question
    private String name;
    public X() {}
    // Alt+Shift+S > Generate Getters and Setters > Select All
}
```

4. Repository XRepository.java (New > Interface)

```
public interface XRepository extends JpaRepository<X, Integer> {}
```

5. Service XService.java

```
@Service
public class XService {
    private final XRepository repo;
    public XService(XRepository repo) { this.repo = repo; }
    public void save(X x) { repo.save(x); }
}
```

6. Controller XController.java

```
@Controller
public class XController {
    private final XService service;
    public XController(XService service) { this.service = service; }

    @GetMapping("/")
    public String form() { return "x-form"; }

    @PostMapping("/save")
    public String save(X x) { service.save(x); return "x-form"; }
}
```

7. JSP views/x-form.jsp

```
<form action="/save" method="post">
    Name: <input name="name"><br>      <- one line per field, name = field name
    <input type="submit">
</form>
```

Hooks: Chain = Entity → Repo → Service → Controller → JSP. Service and Controller have the SAME top (private final + constructor). action="/save" post = @PostMapping("/save"). input name = entity field. int = numbers, double = price/salary/balance, String = text and dates.
Shortcuts: Ctrl+Shift+O imports (choose jakarta) | Alt+Shift+S getters/setters | Ctrl+S save

Q1. Hospital Patient

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **hospital** > Save

Step 2. STS > File > New > Spring Starter Project > Name **PatientApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/hospital
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Patient** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Patient {
    @Id
    private int patientId;
    private String name;
    private int age;
    private String gender;
    private String contactNumber;
    private String email;
    private String address;
    private String bloodGroup;
    public Patient() {}
}
```

Step 6. New > **Interface** > **PatientRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface PatientRepository extends JpaRepository<Patient, Integer> {}
```

Step 7. New > Class > **PatientService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class PatientService {
    private final PatientRepository repo;
    public PatientService(PatientRepository repo) { this.repo = repo; }
    public void save(Patient p) { repo.save(p); }
}
```

Step 8. New > Class > **PatientController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class PatientController {
    private final PatientService service;
    public PatientController(PatientService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "patient-form"; }
    @PostMapping("/save")
    public String save(Patient p) { service.save(p); return "patient-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **patient-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Patient ID: <input name="patientId"><br>
  Name: <input name="name"><br>
  Age: <input name="age"><br>
  Gender: <input name="gender"><br>
  Contact: <input name="contactNumber"><br>
  Email: <input name="email"><br>
  Address: <input name="address"><br>
  Blood Group: <input name="bloodGroup"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > hospital > Query Tool > **SELECT * FROM patient;** > F5 > row is there

Q2. Library Management

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **library** > Save

Step 2. STS > File > New > Spring Starter Project > Name **BookApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/library
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Book** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Book {
    @Id
    private int bookId;
    private String title;
    private String author;
    private String publisher;
    private String isbn;
    private String edition;
    private int publicationYear;
    private String category;
    private int availableCopies;
    public Book() {}
}
```

Step 6. New > **Interface** > **BookRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface BookRepository extends JpaRepository<Book, Integer> {}
```

Step 7. New > Class > **BookService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class BookService {
    private final BookRepository repo;
    public BookService(BookRepository repo) { this.repo = repo; }
    public void save(Book b) { repo.save(b); }
}
```

Step 8. New > Class > **BookController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class BookController {
    private final BookService service;
    public BookController(BookService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "book-form"; }
    @PostMapping("/save")
    public String save(Book b) { service.save(b); return "book-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **book-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Book ID: <input name="bookId"><br>
  Title: <input name="title"><br>
  Author: <input name="author"><br>
  Publisher: <input name="publisher"><br>
  ISBN: <input name="isbn"><br>
  Edition: <input name="edition"><br>
  Year: <input name="publicationYear"><br>
  Category: <input name="category"><br>
  Copies: <input name="availableCopies"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > library > Query Tool > **SELECT * FROM book;** > F5 > row is there

Q3. Employee Management

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **company** > Save

Step 2. STS > File > New > Spring Starter Project > Name **EmployeeApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/company
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Employee** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Employee {
    @Id
    private int employeeId;
    private String name;
    private int age;
    private String gender;
    private String contactNumber;
    private String email;
    private String address;
    private String department;
    private String designation;
    private String dateOfJoining;
    private double salary;
    public Employee() {}
}
```

Step 6. New > **Interface** > **EmployeeRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {}
```

Step 7. New > Class > **EmployeeService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class EmployeeService {
    private final EmployeeRepository repo;
    public EmployeeService(EmployeeRepository repo) { this.repo = repo; }
    public void save(Employee e) { repo.save(e); }
}
```

Step 8. New > Class > **EmployeeController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class EmployeeController {
    private final EmployeeService service;
    public EmployeeController(EmployeeService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "employee-form"; }
    @PostMapping("/save")
    public String save(Employee e) { service.save(e); return "employee-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **employee-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Employee ID: <input name="employeeId"><br>
  Name: <input name="name"><br>
  Age: <input name="age"><br>
  Gender: <input name="gender"><br>
  Contact: <input name="contactNumber"><br>
  Email: <input name="email"><br>
  Address: <input name="address"><br>
  Department: <input name="department"><br>
  Designation: <input name="designation"><br>
  Joining Date: <input name="dateOfJoining"><br>
  Salary: <input name="salary"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > company > Query Tool > **SELECT * FROM employee;** > F5 > row is there

Q4. Product Inventory

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **inventory** > Save

Step 2. STS > File > New > Spring Starter Project > Name **ProductApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/inventory
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Product** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Product {
    @Id
    private int productId;
    private String productName;
    private String category;
    private String brand;
    private String description;
    private double price;
    private int quantity;
    private String supplierName;
    private String manufacturingDate;
    private String expiryDate;
    public Product() {}
}
```

Step 6. New > **Interface** > **ProductRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface ProductRepository extends JpaRepository<Product, Integer> {}
```

Step 7. New > Class > **ProductService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class ProductService {
    private final ProductRepository repo;
    public ProductService(ProductRepository repo) { this.repo = repo; }
    public void save(Product p) { repo.save(p); }
}
```

Step 8. New > Class > **ProductController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "product-form"; }
    @PostMapping("/save")
    public String save(Product p) { service.save(p); return "product-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **product-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Product ID: <input name="productId"><br>
  Name: <input name="productName"><br>
  Category: <input name="category"><br>
  Brand: <input name="brand"><br>
  Description: <input name="description"><br>
  Price: <input name="price"><br>
  Quantity: <input name="quantity"><br>
  Supplier: <input name="supplierName"><br>
  Mfg Date: <input name="manufacturingDate"><br>
  Expiry Date: <input name="expiryDate"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > inventory > Query Tool > **SELECT * FROM product;** > F5 > row is there

Q5. Online Banking

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **bank** > Save

Step 2. STS > File > New > Spring Starter Project > Name **AccountApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/bank
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Account** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Account {
    @Id
    private int accountId;
    private String accountNumber;
    private String customerName;
    private int age;
    private String gender;
    private String contactNumber;
    private String email;
    private String address;
    private String accountType;
    private double accountBalance;
    public Account() {}
}
```

Step 6. New > **Interface** > **AccountRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface AccountRepository extends JpaRepository<Account, Integer> {}
```

Step 7. New > Class > **AccountService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class AccountService {
    private final AccountRepository repo;
    public AccountService(AccountRepository repo) { this.repo = repo; }
    public void save(Account a) { repo.save(a); }
}
```

Step 8. New > Class > **AccountController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class AccountController {
    private final AccountService service;
    public AccountController(AccountService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "account-form"; }
    @PostMapping("/save")
    public String save(Account a) { service.save(a); return "account-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **account-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Account ID: <input name="accountId"><br>
  Account No: <input name="accountNumber"><br>
  Name: <input name="customerName"><br>
  Age: <input name="age"><br>
  Gender: <input name="gender"><br>
  Contact: <input name="contactNumber"><br>
  Email: <input name="email"><br>
  Address: <input name="address"><br>
  Type: <input name="accountType"><br>
  Balance: <input name="accountBalance"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > bank > Query Tool > **SELECT * FROM account;** > F5 > row is there

Q6. Course Management

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **college** > Save

Step 2. STS > File > New > Spring Starter Project > Name **CourseApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/college
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Course** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Course {
    @Id
    private int courseId;
    private String courseCode;
    private String courseName;
    private String department;
    private String facultyName;
    private int credits;
    private int semester;
    private int courseCapacity;
    public Course() {}
}
```

Step 6. New > **Interface** > **CourseRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface CourseRepository extends JpaRepository<Course, Integer> {}
```

Step 7. New > Class > **CourseService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class CourseService {
    private final CourseRepository repo;
    public CourseService(CourseRepository repo) { this.repo = repo; }
    public void save(Course c) { repo.save(c); }
}
```

Step 8. New > Class > **CourseController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class CourseController {
    private final CourseService service;
    public CourseController(CourseService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "course-form"; }
    @PostMapping("/save")
    public String save(Course c) { service.save(c); return "course-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **course-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Course ID: <input name="courseId"><br>
  Code: <input name="courseCode"><br>
  Name: <input name="courseName"><br>
  Department: <input name="department"><br>
  Faculty: <input name="facultyName"><br>
  Credits: <input name="credits"><br>
  Semester: <input name="semester"><br>
  Capacity: <input name="courseCapacity"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > college > Query Tool > **SELECT * FROM course;** > F5 > row is there

Q7. Blood Donor Management

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **blooddb** > Save

Step 2. STS > File > New > Spring Starter Project > Name **DonorApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/blooddb
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Donor** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Donor {
    @Id
    private int donorId;
    private String name;
    private int age;
    private String gender;
    private String bloodGroup;
    private String contactNumber;
    private String email;
    private String address;
    private String lastDonationDate;
    private String healthStatus;
    public Donor() {}
}
```

Step 6. New > **Interface** > **DonorRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface DonorRepository extends JpaRepository<Donor, Integer> {}
```

Step 7. New > Class > **DonorService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class DonorService {
    private final DonorRepository repo;
    public DonorService(DonorRepository repo) { this.repo = repo; }
    public void save(Donor d) { repo.save(d); }
}
```

Step 8. New > Class > **DonorController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class DonorController {
    private final DonorService service;
    public DonorController(DonorService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "donor-form"; }
    @PostMapping("/save")
    public String save(Donor d) { service.save(d); return "donor-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **donor-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Donor ID: <input name="donorId"><br>
  Name: <input name="name"><br>
  Age: <input name="age"><br>
  Gender: <input name="gender"><br>
  Blood Group: <input name="bloodGroup"><br>
  Contact: <input name="contactNumber"><br>
  Email: <input name="email"><br>
  Address: <input name="address"><br>
  Last Donation: <input name="lastDonationDate"><br>
  Health Status: <input name="healthStatus"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > blooddb > Query Tool > **SELECT * FROM donor;** > F5 > row is there

Q8. Pharmacy Management

Step 1. pgAdmin 4 > Servers > PostgreSQL (password) > right-click Databases > Create > Database > **pharmacy** > Save

Step 2. STS > File > New > Spring Starter Project > Name **MedicineApp**, package com.example.demo > Next > tick **Spring Web, Spring Data JPA, PostgreSQL Driver** > Finish

Step 3. pom.xml > inside <dependencies> add > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

Step 4. src/main/resources/application.properties > add > Ctrl+S

```
spring.datasource.url=jdbc:postgresql://localhost:5432/pharmacy
spring.datasource.username=postgres
spring.datasource.password=yourpass
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

Step 5. Right-click **com.example.demo** > New > Class > **Medicine** > type > Ctrl+Shift+O > Alt+Shift+S > Generate Getters and Setters > Select All > Ctrl+S

```
@Entity
public class Medicine {
    @Id
    private int medicineId;
    private String medicineName;
    private String category;
    private String manufacturer;
    private String batchNumber;
    private double price;
    private int quantity;
    private String manufacturingDate;
    private String expiryDate;
    private String supplierName;
    public Medicine() {}
}
```

Step 6. New > **Interface** > **MedicineRepository** > Ctrl+Shift+O > Ctrl+S

```
public interface MedicineRepository extends JpaRepository<Medicine, Integer> {}
```

Step 7. New > Class > **MedicineService** > Ctrl+Shift+O > Ctrl+S

```
@Service
public class MedicineService {
    private final MedicineRepository repo;
    public MedicineService(MedicineRepository repo) { this.repo = repo; }
    public void save(Medicine m) { repo.save(m); }
}
```

Step 8. New > Class > **MedicineController** > Ctrl+Shift+O > Ctrl+S

```
@Controller
public class MedicineController {
    private final MedicineService service;
    public MedicineController(MedicineService service) { this.service = service; }
    @GetMapping("/")
    public String form() { return "medicine-form"; }
    @PostMapping("/save")
    public String save(Medicine m) { service.save(m); return "medicine-form"; }
}
```

Step 9. Lower **src** > right-click **main** > New > Folder > **webapp/WEB-INF/views**. Right-click views > New > File > **medicine-form.jsp** > Ctrl+S

```
<form action="/save" method="post">
  Medicine ID: <input name="medicineId"><br>
  Name: <input name="medicineName"><br>
  Category: <input name="category"><br>
  Manufacturer: <input name="manufacturer"><br>
  Batch No: <input name="batchNumber"><br>
  Price: <input name="price"><br>
  Quantity: <input name="quantity"><br>
  Mfg Date: <input name="manufacturingDate"><br>
  Expiry Date: <input name="expiryDate"><br>
  Supplier: <input name="supplierName"><br>
  <input type="submit">
</form>
```

Step 10. Right-click project > Run As > **Spring Boot App** (console: Tomcat started on port 8080) > browser **localhost:8080/** > fill all boxes > Submit > pgAdmin > pharmacy > Query Tool > **SELECT * FROM medicine;** > F5 > row is there